

Components & Technology Stack

Every real component used to build POLAR-EMS, grouped by where it runs. Nothing in the running product depends on a JS framework, a charting library, or a cloud ML API — the only non-trivial dependency anywhere is scikit-learn, used strictly offline for training.

Frontend

RUNS IN THE BROWSER

HTML / CSS / JavaScript

No framework (no React/Vue), no build step. One self-contained `index.html` runs the entire simulation client-side.

Three.js

`three@0.160.0`, loaded via CDN import map — the cinematic hero scene and 3D digital twin.

Native

`<video>`

The looping Antarctic station background, no player library.

Hand-drawn inline SVG

Every chart — line/bar charts, the power-flow diagram, the digital twin — drawn directly, no charting library.

Google Fonts

Big Shoulders Display (headings), IBM Plex Sans (body), IBM Plex Mono (data/labels).

Backend

PYTHON SERVICE

Python 3.11

Runtime for the entire backend.

FastAPI

The REST API framework — `/api/telemetry`, `/api/forecast`, `/api/green-audit`, and more.

Uvicorn

The ASGI server running the FastAPI app.

Pydantic

Request/response validation for the API.

`requests` + `rich`

Power `cli.py`, the terminal client — `rich` renders its menu, tables and panels.

Machine Learning

scikit-learn

`GradientBoostingRegressor` (150 trees), trained offline with a genuine held-out train/test split.

NumPy

Numerical operations for training and export.

Custom JS tree-traversal engine

Reimplements the trained model's inference in the browser — the trained model is exported to JSON and walked in plain JS, verified bit-for-bit against scikit-learn's own output before shipping. No ML library needed client-side.

Custom online linear regression

Widrow-Hoff / LMS rule, hand-implemented in JS — trains live from the running simulation, one gradient step per tick.

Deployment & Infrastructure

Git / GitHub

Single source of truth — `sayansaunth-lang/polar-ems-sih`.

Vercel

Hosts the frontend as a static CDN deployment.

Render

Hosts the backend — containerized, HTTPS, auto-deploys on push (`render.yaml`).

Dev Tooling

USED TO BUILD & VERIFY, NOT PART OF THE RUNNING APP

VS Code

`.vscode/launch.json` run configs for the backend and CLI client.

Python venv + pip

Dependency isolation, pinned in `requirements.txt`.

Headless Chrome

Renders the documentation, reports and slide graphics from HTML to PDF/PNG.

pypdf

Verifies generated PDFs actually contain the right content before they're shipped.

POLAR-EMS · Components & Technology Stack · reflects the current running codebase